How Is Model Context Protocol (MCP) Transforming the Future of AI-Tool Integration?

Understanding How AI Applications Connect to External Tools, Data Sources, and Services

Prof. Mohammad Saleh

Students :

Setareh Kashanchi

Fatemeh Yahyavi

Major : Computer Engineering



Date : 2026 October 12


Presentation Roadmap


Section 1


Section 2


Section 3


Section 4


Section 5


Section 6


Section 7


Section 8


Conclusion & Resources

This presentation explores the Model Context Protocol through seven major sections:

1. What Is MCP, and Why Does Modern AI Need It?
2. How Does MCP Work Behind the Scenes?
3. How Does MCP Differ from Traditional APIs?
4. How Does MCP Empower AI Agents?
5. What Security Challenges Does MCP Introduce?
6. How Is MCP Used in Real-World Applications?
7. What Does the Future Hold for MCP and AI-Tool Integration?

INTRODUCTION TO MCP

What Is Model Context Protocol?

Model Context Protocol (MCP) is an open protocol designed to standardize how AI applications connect to external tools, data sources, and services.

MCP defines a common communication structure that allows AI applications to discover available capabilities, request operations, and receive information from external systems.

Key Characteristics:

Open Protocol: MCP is publicly specified and can be implemented by different developers and organizations.

Standardized Communication: It provides common rules for exchanging information between compatible applications and servers.

Tool Integration: It enables AI applications to interact with external functionality.

Data Access: It supports structured access to contextual information.

Interoperability: It helps different AI applications reuse compatible integrations.

Important: MCP is not an AI model, a programming language, or a database. It is a protocol for connecting AI applications to external capabilities.



Why Was MCP Introduced?

Modern AI systems are expected to perform tasks beyond generating text.

Users increasingly want AI applications to retrieve documents, access databases, interact with development tools, and perform operations through external services.

Before standardized integration approaches, developers frequently had to create custom connections for each application and service.

Major Challenges:

Different services use different APIs and communication formats.

Custom integrations require additional development effort.

Maintaining multiple integrations becomes increasingly difficult.

Changes in external services may require code modifications.

Integrations developed for one application may not be reusable elsewhere.

MCP was introduced to reduce these challenges by defining a shared protocol for AI-tool communication.



The Problem MCP Aims to Solve

One of the main problems MCP addresses is fragmented AI-tool integration.

When multiple AI applications need access to multiple external services, developers may need to build and maintain numerous custom connections.

Without MCP

AI applications may require service-specific integration logic.

Tool definitions and communication formats can vary.

Integration code may be duplicated across applications.

Maintenance becomes more difficult as systems expand.

With MCP

Compatible applications can communicate with MCP servers through a shared protocol.

Tools and resources can be discovered using standardized methods.

Integrations can be reused across different MCP-compatible hosts.

Developers can focus more on application functionality.

MCP does not eliminate the need for APIs or server-specific implementation logic. It standardizes the communication layer used by MCP-compatible applications.

HOW MCP WORKS

Overview of MCP Architecture

MCP follows a client-server architecture designed to connect AI applications with external capabilities.

The architecture includes three primary components:

1. MCP Host

The AI application responsible for managing the user experience, coordinating AI interactions, and managing MCP connections.

2. MCP Client

A protocol component within the host that establishes and maintains communication with an MCP server.

3. MCP Server

A program that exposes capabilities such as tools, resources, and prompts through MCP.

Basic Architecture:



The MCP server processes supported requests and returns results through the client.

The host can then incorporate the results into the AI application's response.

MCP Hosts, Clients, and Servers

Although MCP hosts, clients, and servers cooperate, each component has distinct responsibilities.

MCP Host Responsibilities:

1. Managing user interactions and application workflows
2. Coordinating communication with AI models
3. Managing MCP client connections
4. Controlling tool execution and user permissions

MCP Client Responsibilities:

1. Establishing connections to MCP servers
2. Exchanging protocol messages
3. Discovering server capabilities
4. Sending requests and receiving responses

MCP Server Responsibilities:

1. Advertising available capabilities
2. Processing supported requests
3. Executing tool operations
4. Providing resources and prompt templates
5. Returning structured results

A single MCP host may connect to multiple MCP servers through separate MCP client connections.



MCP Core Primitives: Tools, Resources, and Prompts

MCP defines three important server-side primitives that provide different capabilities to AI applications.

1. TOOLS

Tools are executable operations that applications can request.
Examples include searching a database, performing calculations, or calling external APIs.

2. RESOURCES

Resources provide contextual information that applications can access.
Examples include documents, source code files, and application records.

3. PROMPTS

Prompts are reusable templates that help structure interactions with AI models.
Examples include code review templates and document summarization templates.

4. KEY DIFFERENCE

Resources provide information.
Prompts provide reusable interaction structures.
Not every MCP server must implement all three primitives.

Tool Discovery and Tool Calling

Tool discovery allows MCP clients to identify tools exposed by connected MCP servers.

A tool definition typically includes:

- Tool name
- Description of its functionality
- Input schema
- Expected input parameters
- Optional output information

After discovering available tools, the AI application can select an appropriate operation for a user's request.

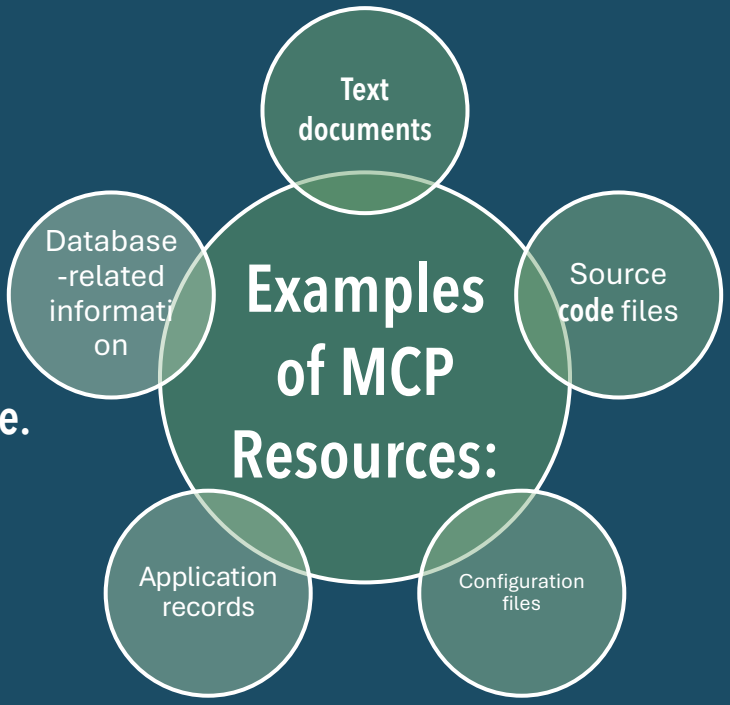
Example:
User request:
"What classes do I have on Tuesday?"
Available tool:
get_classes_by_day

Tool arguments:
{"day": "Tuesday"}

The MCP client sends a tool-call request to the server.
The server executes the operation and returns the result.
This process is known as Tool Calling.

MCP Resources

Resources allow MCP servers to expose contextual information that applications can read.
Resources are identified using URIs.
A URI, or Uniform Resource Identifier, identifies a particular resource.



Depending on the server's capabilities, clients can discover and read available resources.

Resources vs. Tools:

Resources primarily provide information, while tools expose callable operations.

For example, reading a document may be supported through a resource, while modifying that document may be implemented through a tool.

Access to resources should be controlled according to appropriate permissions.



MCP Prompts

MCP servers can provide reusable prompt templates to support structured AI interactions.

A prompt template contains predefined content designed for a particular task.

Prompts may accept arguments that customize their content.

Examples:

Code Review Prompt

A template for reviewing source code and identifying possible improvements.

Document Summary Prompt

A template for summarizing documents using a consistent structure.

Technical Explanation Prompt

A template for explaining technical concepts.

Benefits of MCP Prompts:

Reusable instructions

Consistent interaction structures

Reduced repetition

Easier implementation of predefined workflows

Prompts provide templates rather than automatically authoritative instructions.



MCP Communication Mechanisms

MCP uses structured messages based on JSON-RPC 2.0.

JSON-RPC defines how requests, responses, and notifications are represented using JSON.

MCP supports different transport mechanisms for exchanging protocol messages.

1. Standard Input/Output (stdio)

Commonly used for communication between local processes.

2. Streamable HTTP

Uses HTTP-based communication and supports networked MCP server deployments.

Important Distinction:

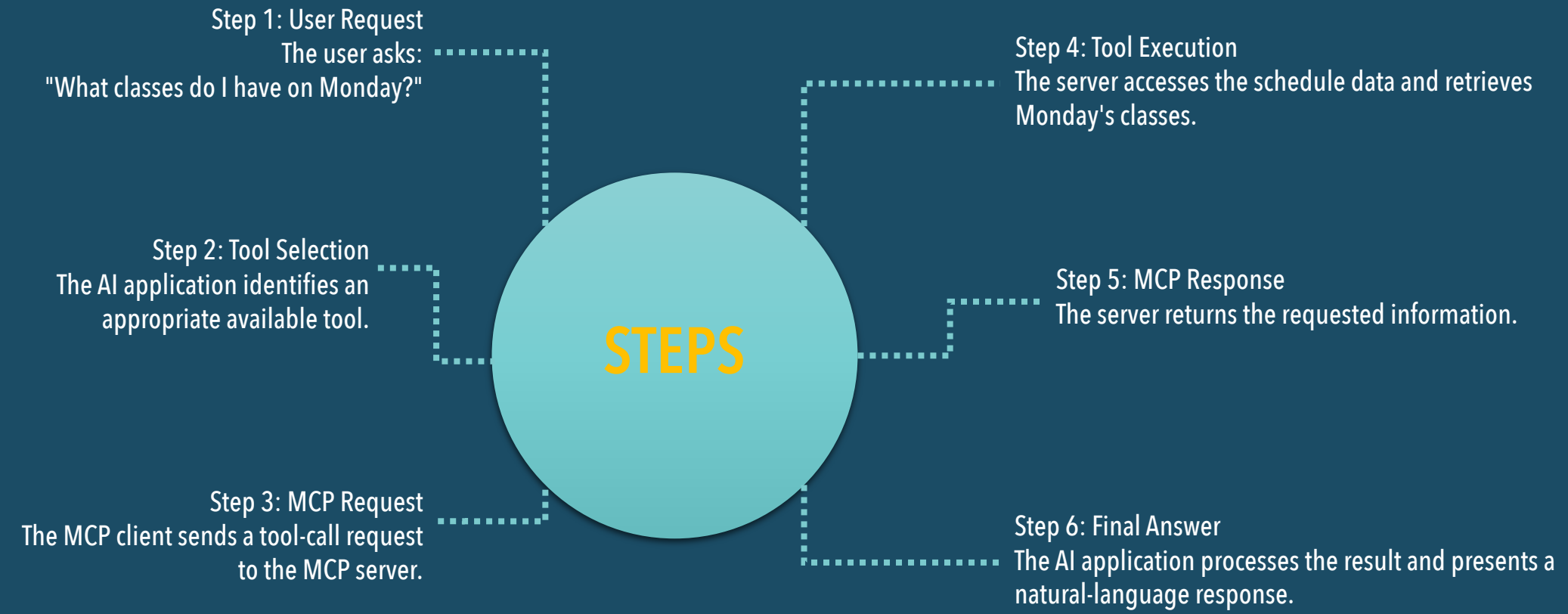
JSON-RPC defines the structure of protocol messages.

Transport mechanisms determine how those messages are transmitted.

This separation allows MCP to operate across different environments.

Step-by-Step MCP Workflow

Consider an AI assistant connected to a university schedule system.



MCP AND APIs

What Is an API?

API stands for Application Programming Interface.

An API is a defined interface that allows software components to communicate and interact with one another.

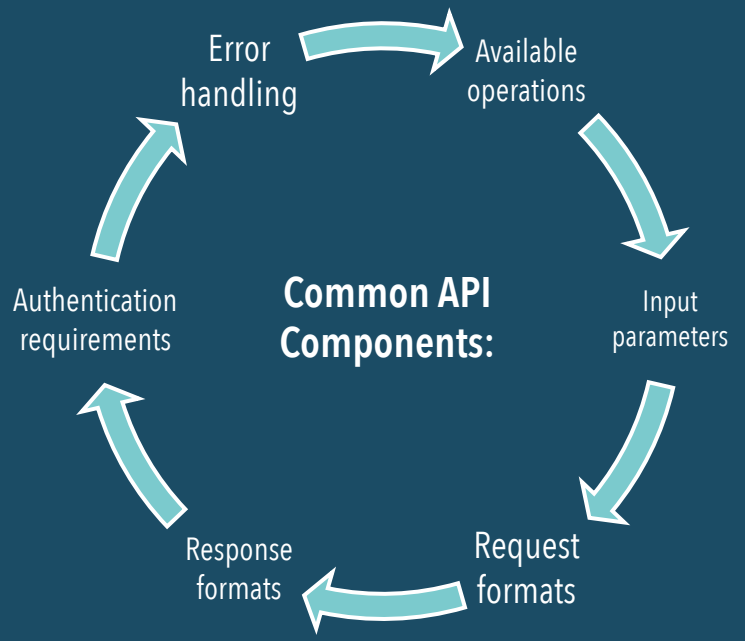
APIs specify how applications can request operations or access information provided by another system.

Example:

A weather application can use a weather API to request temperature information.

The API receives the request and returns the relevant data.

An API is a software interface, not necessarily a graphical user interface.





MCP vs. Traditional APIs

Traditional APIs and MCP serve related but different purposes.

Traditional APIs:

Expose operations and data from specific software services.
Often require service-specific integration code.
Can use different protocols and data formats.
Are commonly designed for general software communicat

VS

MCP:

Defines a common protocol for AI application integrations.
Supports standardized capability discovery.
Provides structured access to tools, resources, and prompts.
Can expose functionality implemented through existing APIs.

Key Difference:

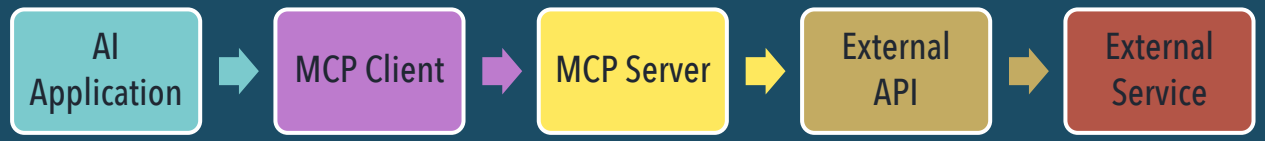
APIs define how software services expose functionality.

MCP standardizes how AI applications discover and interact with capabilities exposed by MCP servers.

MCP complements APIs rather than replacing them.

How MCP Connects AI Applications to APIs

MCP servers can act as adapters between AI applications and existing external APIs.



Example: Project Management Integration

A user asks an AI assistant to retrieve open project issues.

The AI application selects the appropriate MCP tool.

The MCP client sends a request to the MCP server.

The MCP server communicates with the project management API.

The API returns the requested information.

The MCP server returns the result to the AI application.

This architecture allows existing APIs to be used through a standardized AI integration protocol.

MCP AND AI AGENTS

What are AI Agents?

AI agents are AI-driven systems designed to pursue goals by selecting actions, using tools, and interacting with their environment.

Unlike systems that only generate a single response, agents may perform multiple steps to complete a task.

Example:

An AI agent assigned to analyze a software project may:

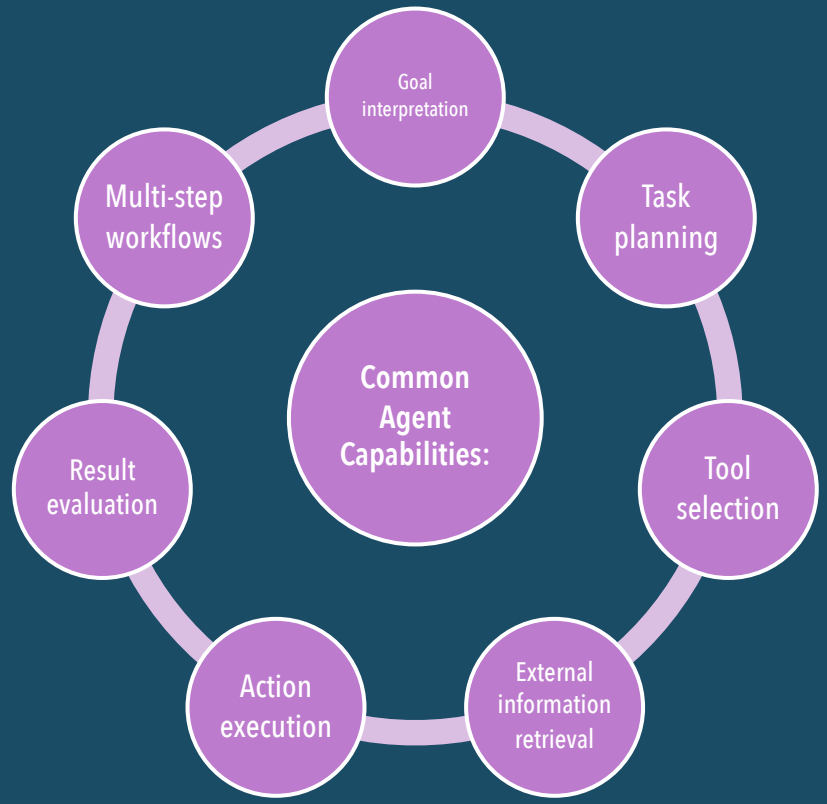
Retrieve repository information.

Examine relevant source files.

Identify potential issues.

Generate a technical report.

Agent capabilities depend on the underlying model, tools, orchestration logic, and system design.



Traditional Chatbots vs. AI Agents

Traditional APIs and MCP serve related but different purposes.

Traditional Chatbots

Primarily focus on conversational responses.
Often operate through individual question-answer interactions.
May have limited access to external systems.
Do not necessarily perform multi-step actions.

VS

AI Agents

Can pursue goals through multiple steps.
May use external tools and services.
Can incorporate information from previous actions.
May execute tasks involving several systems.
Require orchestration, permissions, and error handling.

Example:

Chatbot:

"Here are the steps to create a project report."

AI Agent:

"I retrieved the project data, analyzed the results, and prepared a report."

Modern chatbots can also use tools, so the distinction depends on actual capabilities rather than product labels.



How MCP Enables AI Agents to Use Tools

AI agents often need external capabilities to complete complex tasks.

MCP provides a standardized way for agent-based applications to discover and invoke tools.

Typical Workflow:

- The agent receives a task.
- The host identifies available tools through MCP.
- The agent selects an appropriate action.
- The host requests tool execution through an MCP client.
- The MCP server performs the operation.
- The result is returned to the agent workflow.
- The agent may continue with another action.

Benefits:

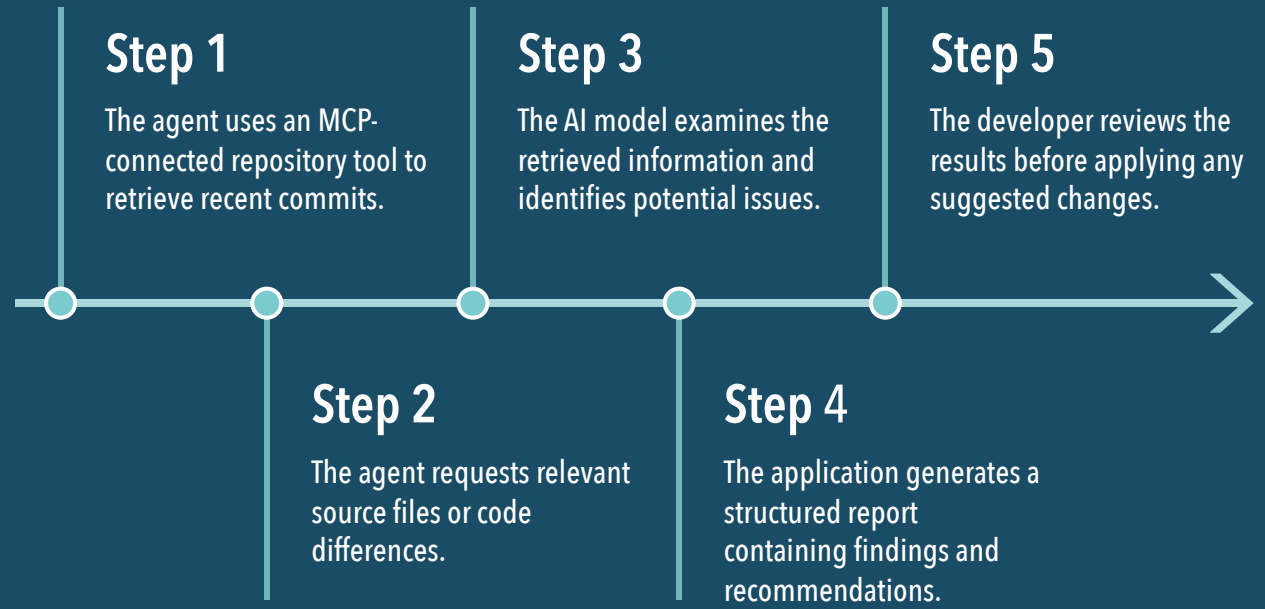
- Standardized tool integration
- Reusable tool connections
- Access to multiple external systems
- Support for multi-step workflows
- Reduced dependence on custom integration formats

Practical Example of an MCP-Powered AI Agent

Scenario: AI Software Development Assistant

A developer asks:

"Analyze the recent changes in my repository and identify possible problems."



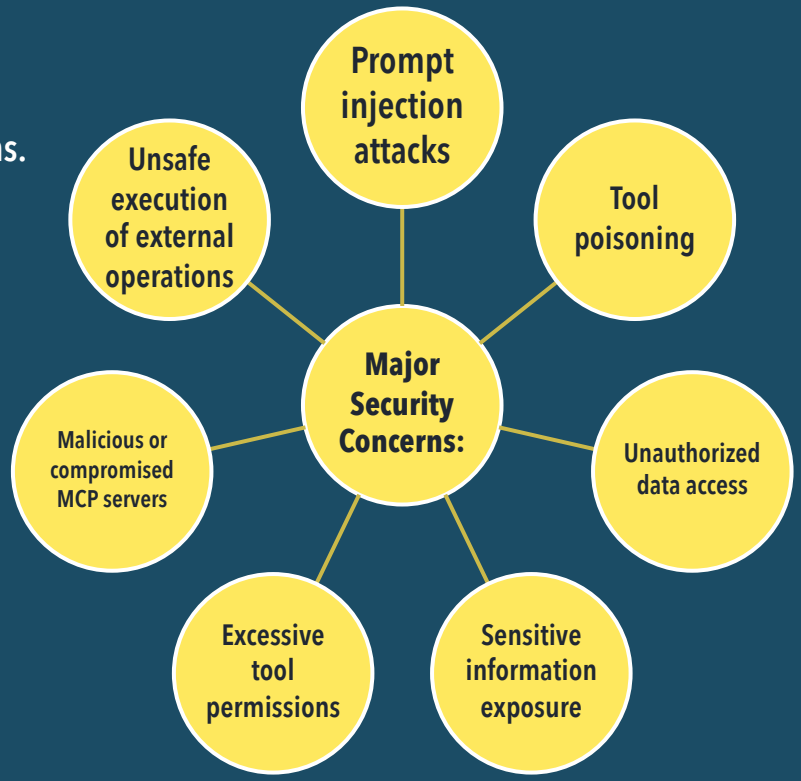
Key Idea: MCP provides tool connectivity, while the agent workflow coordinates the overall task.

SECURITY CHALLENGES IN MCP

Security Challenges in MCP

Connecting AI applications to external tools introduces additional security risks.

An AI system with tool access may interact with sensitive information or perform actions that affect real systems.



Important Principle:

MCP provides a communication protocol, but secure behavior depends on the implementation of the host, client, server, and connected services.



Prompt Injection Attacks

Prompt injection occurs when untrusted content attempts to influence an AI system's behavior by introducing malicious instructions.

These **instructions** may appear inside:

- Retrieved documents
- Web pages
- Tool responses
- Database content
- External messages

Example:
An AI assistant retrieves a document containing legitimate information.
The document also includes malicious instructions asking the assistant to reveal confidential information.

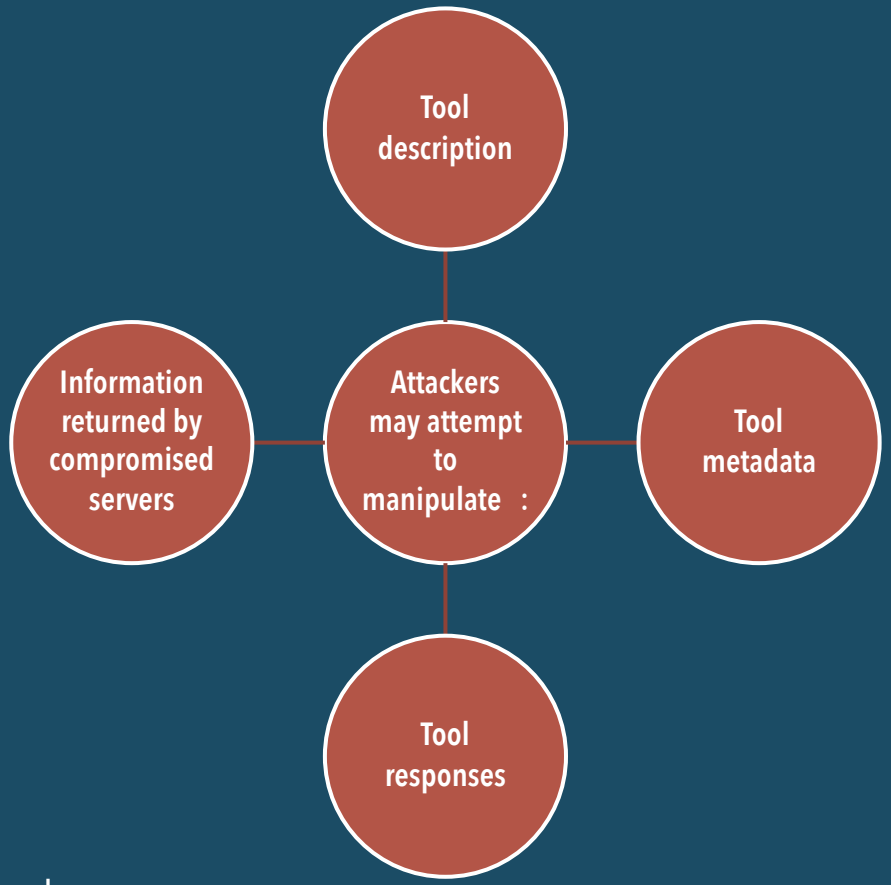
If the assistant treats those instructions as authoritative, it may behave incorrectly.

Potential Defenses:

- Treat external content as untrusted data.
- Separate instructions from retrieved information.
- Restrict sensitive tool access.
- Require approval for high-risk operations.
- Validate actions before execution.

Tool Poisoning Attacks

Tool poisoning is a security threat in which malicious or misleading tool-related content influences an AI application's behavior.



Example:
A malicious tool description contains hidden instructions intended to influence the AI assistant.

The assistant may incorrectly trust the description and perform an unintended action.

Possible Consequences:

- Unauthorized tool execution
- Sensitive data exposure
- Manipulated outputs
- Incorrect tool selection
- Unsafe workflow execution

Mitigation Strategies:

- Use trusted MCP servers.
- Review tool definitions.
- Apply least-privilege access.
- Monitor sensitive operations.
- Validate server responses.



Authentication, Authorization, and Data Privacy

Security in MCP-based systems requires appropriate identity verification and access control.

Authentication

Authentication verifies the identity of a user, application, or service.

Authorization

Authorization determines which resources and operations an authenticated identity is permitted to access.

Data Privacy

Sensitive information must be protected against unauthorized disclosure.

Recommended Security Practices:

- Use secure authentication mechanisms.
- Apply least-privilege permissions.
- Protect credentials and access tokens.
- Restrict access to confidential data.
- Require confirmation for sensitive actions.
- Maintain appropriate audit logs.
- Use secure communication channels.



MCP Security Best Practices

Secure MCP deployments require protection at multiple layers.

1. Trusted Servers

Connect only to MCP servers that have been appropriately reviewed.

2. Input Validation

Validate tool arguments before executing operations.

3. Access Control

Restrict tools and resources according to user permissions.

4. Human Approval

Require confirmation before performing destructive or sensitive actions.

5. Credential Protection

Store credentials securely and avoid exposing them in logs or model context.

6. Monitoring

Record relevant security events and investigate suspicious behavior.

7. Regular Updates

Keep MCP implementations and dependencies updated.

Security is a system-wide responsibility, not a feature guaranteed by MCP alone.

REAL-WORLD APPLICATIONS

MCP in Software Development

MCP can support AI-powered software development tools by providing access to development environments and related services.

Benefits:

- Easier integration with development services
- Reusable tool connections
- More context-aware coding assistance
- Support for multi-step development workflows

Potential Applications:

Repository Management	Accessing repository information, branches, commits, and issues.
Code Analysis	Retrieving source files for review and analysis.
Development Documentation	Accessing technical documentation and project information.
Issue Tracking	Retrieving or updating development tasks through authorized tools.
Development Automation	Supporting repetitive workflows through connected tools.



MCP in Databases, Cloud Services, and Automation

MCP can connect AI applications to a variety of external systems.

1

Databases and Data Analysis

Querying authorized databases

Retrieving structured information

Supporting analytical workflows

Generating reports from retrieved data

2

Cloud Services

Accessing supported cloud applications

Retrieving service information

Interacting with approved cloud operations

3

File and Document Management

Searching documents

Reading files

Retrieving contextual information

Supporting document workflows

4

Workflow Automation

Connecting multiple tools

Executing approved operations

Supporting multi-step business processes

Important: Available capabilities depend on the specific MCP server and its permissions.

THE FUTURE OF MCP

Benefits and Current Limitations of MCP

MCP provides important advantages for AI-tool integration, but it also introduces technical challenges.

Major Benefits:
Standardized communication
Reusable integrations
Tool and resource discovery
Improved interoperability
Support for local and remote tools
Easier integration of external capabilities into AI applications

Current Limitations and Trade-offs:
Current Limitations and Trade-offs:
Server implementation and maintenance requirements
Potential communication latency
Security and permission management challenges
Compatibility considerations
Dependence on the reliability of connected services

Important: MCP does not guarantee accurate model decisions, secure tool execution, or correct external data.



The Future of MCP and AI Agents

As AI systems become more capable of interacting with external environments, standardized tool integration may become increasingly important.

MCP has the potential to support more interoperable AI ecosystems.

Potential Future Directions:

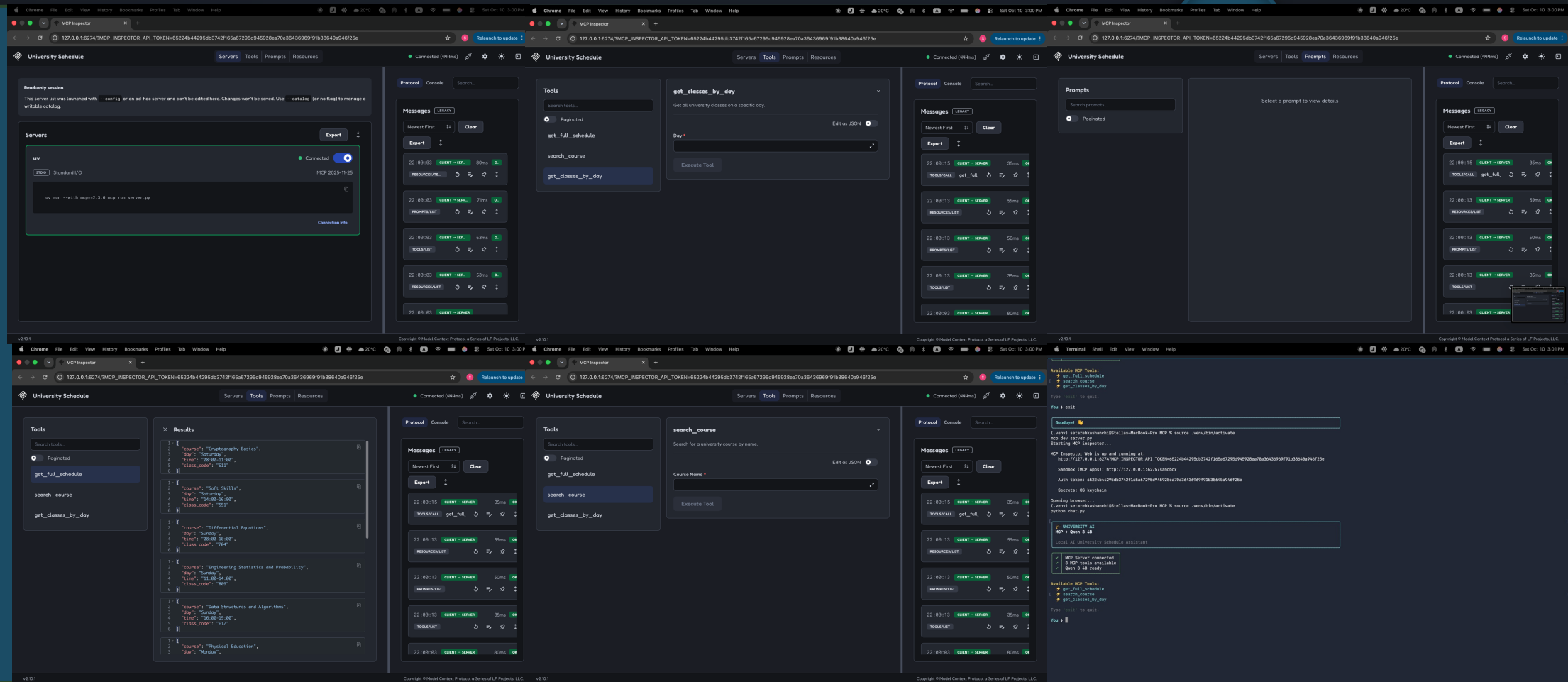
1. More Capable AI Agents : Agents may use a wider range of specialized tools and services.
2. Greater Interoperability : Compatible AI applications may reuse integrations across different environments.
3. Enterprise AI Integration : Organizations may connect AI applications to internal systems through controlled interfaces.
4. Advanced Workflow Automation : AI systems may coordinate increasingly complex workflows involving multiple tools.
5. Improved Security and Governance : Future development may place greater emphasis on permissions, auditing, and secure delegation.

Remaining Challenges:
Security, scalability, reliability, performance, and standard adoption.

These developments represent potential directions rather than guaranteed outcomes.

PRACTICAL DEMONSTRATION

To be developed separately





Questions:

Question 1: Why is MCP useful when an AI application needs to connect to multiple external tools and services?

Question 2: Imagine an AI assistant needs to retrieve information from GitHub. How can MCP help it discover and use the appropriate tool?



Answers:

Question 1: Why is MCP useful when an AI application needs to connect to multiple external tools and services?

Answer: MCP provides a standardized way for AI applications to discover and interact with external tools, reducing integration complexity and improving interoperability.

Question 2: Imagine an AI assistant needs to retrieve information from GitHub. How can MCP help it discover and use the appropriate tool?

Answer: An MCP client can discover available tools from an MCP server. The AI application can then select an appropriate tool, request its execution, and use the returned information to respond to the user.

Conclusion and References

Conclusion

Model Context Protocol represents an important approach to standardizing communication between AI applications and external tools.

Key Takeaways:

MCP is an open protocol for AI-tool integration.
Its architecture includes hosts, clients, and servers.
MCP servers can expose tools, resources, and prompts.
MCP complements existing APIs rather than replacing them.
MCP can support tool-using AI assistants and agents.
Security, authentication, authorization, and privacy remain essential.
Standardized tool integration may play an important role in future AI systems.

Final Thought:

The value of MCP is not that it makes AI models intelligent by itself, but that it provides a standardized way for AI applications to interact with capabilities beyond the model.



Conclusion and References

References:

Official Model Context Protocol Documentation:

https://modelcontextprotocol.io/

MCP Technical Specification:

https://modelcontextprotocol.io/specification/

Official MCP GitHub Organization:

https://github.com/modelcontextprotocol

MCP Python SDK:

https://github.com/modelcontextprotocol/python-sdk

Anthropic: Introducing the Model Context Protocol:

https://www.anthropic.com/news/model-context-protocol